Information retrieval: dense retrieval

SciFact, and the same judgements as last week

LECTURE 20 LECTURE NOTES

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we left off

BM25, with no training at all, scored **0.6611** NDCG@10 on SciFact. And it failed in a way no parameter could fix:

> “In mice, P. chabaudi parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers.”

Its one relevant abstract shares two words with it — `in` and `to` — and BM25 ranks that abstract ***X* 4,999** of 5,183.

TWO SEPARATE GAPS, NEEDING TWO SEPARATE ANSWERS

Ranked too low: recall@10 is 77.8% but recall@100 is 88.5%. Those documents were found and misplaced.

Never found: the rest. No re-ordering of what BM25 returned can reach them.

The vocabulary of the problem, once more

Three words get used interchangeably and should not be:

Retrieval	producing candidates from the whole corpus. Judged on recall at depth
Ranking	ordering a fixed candidate set. Judged on NDCG, MRR, P@1
Relevance	a property of a pair, decided by a person, recorded in the judgements

A method improves one of the first two. Claims that mix them are how “our re-ranker improves retrieval” gets written, which is a sentence that cannot be true.

Our numbers: retrieval, 0.8852 against 0.9250; ranking, 0.6611 against 0.6451. Two different questions, and the two systems trade places between them.

Today

1. Vocabulary mismatch, and why an index of strings cannot solve it (10 min)
2. **Bi-encoders** — encode once, retrieve by inner product; and in-batch negatives (30 min)
3. **Measured** against BM25, on the same judgements (20 min)
4. Cross-encoders and re-ranking; hybrid retrieval; approximate nearest neighbours (30 min)

No derivation today. The mathematics of this lecture is the dot product, and you have had it since Lecture 8.

What you should be able to do afterwards

- Say what a bi-encoder computes and why it can be indexed in advance
- Say what a cross-encoder computes and why it cannot
- Explain in-batch negatives, and what the batch size actually buys
- State the first-stage ceiling argument, and compute it from a recall curve
- Read a recall/latency curve for an approximate index and choose a point on it
- Say, from a table, when a neural retriever is not worth deploying

FIRST TEN MINUTES

What an index of strings cannot do

10 minutes

The mismatch, restated as a requirement

Last lecture measured it: a relevant abstract is missing **40.9%** of its claim's words on average, and 28.6% of judged pairs share fewer than half.

- "heart attack" and "myocardial infarction"
- "lower numbers" and "reduced inoculum"
- A mechanism described rather than named

> **WHAT IS NEEDED**
>
> A representation in which **two texts that mean the same thing are close**, whether or not they share a word. Which is precisely what Lectures 17 and 18 built, and why Part V comes after Part IV.

The three repairs that do not work

Repair	Why it falls short
Stemming	merges word forms, not concepts — nothing links *infarction* to *attack*
A synonym dictionary	someone must write it, per domain, and it cannot cover paraphrase
Query expansion	adds terms from the top results, so it needs the answer to already be near the top — exactly what fails here

Each is a patch on the same assumption: that relevance is visible in shared strings. The assumption is what has to go.

This is the shape of every move from feature engineering to learned representations in this course. The patches are not stupid; they are bounded, and the bound is the assumption.

What a lexical index gets right

Before replacing it, be fair to it. BM25 is **exact** where the words do match:

- A rare technical term — a gene name, a drug, an identifier — is matched exactly, and an embedding may blur it into its neighbours
- It needs no training data, no GPU and no model version to record
- Its score is **explainable**: the contribution of every term can be printed, as we did last lecture
- A new document is indexed in microseconds; no re-encoding, no re-fitting

None of that is nostalgia. Three of the four are operational requirements that outlive whichever model is fashionable, and they are why BM25 is still in the first stage of systems that also run neural rankers.

What Part IV already gave us

- Lecture 17: a token embedding, and a sentence vector from pooling
- Lecture 18: attention, so a token's vector depends on its context — *bank* in a river sentence and in a finance sentence are no longer the same vector
- Both: a fixed-width vector for a variable-length text

> **THE ONE NEW IDEA IN THIS LECTURE**
>
> Put the **query and the document in the same space**, and make the score a dot product. Everything else today is a consequence of that choice — what can be precomputed, what must be approximated, and what the model is allowed to see.

Which is why there is no derivation: the mathematics is a normalised inner product, and you have had it since Lecture 8.

From tokens to one vector

Lecture 18 left us with one vector per token. A retrieval score needs one vector per *text*, so they must be pooled:

Pooling	What it does
[CLS]	take the reserved first token's vector — only meaningful if the model was trained to put something there
Mean	average the token vectors, masking padding — robust, and what our model uses
Max	component-wise maximum — rarely better, and discards how often a thing was said

✗ Masking the padding is not optional. Mean-pooling *over* the padding tokens makes a text's vector depend on the batch it was encoded in — the same abstract gets two different vectors, and nothing raises an error.

Why the vectors are normalised

Normalise to unit length and the inner product becomes the cosine, which throws away vector *magnitude*. That is deliberate.

- Magnitude in these encoders tracks length and token frequency, not relevance — so an unnormalised dot product quietly prefers long documents, which is the failure BM25's b was invented for
- Normalised scores are bounded in $[-1, 1]$, which makes a threshold transferable between corpora
- And it makes the clustering in the last section legitimate: k-means on the sphere is a meaningful thing to do

The same trap as Lecture 19's length normalisation, arriving from a different direction. Long documents win by default unless something is done about it.

THE METHOD

Bi-encoders

30 minutes · examinable

Encode both sides into one space

$$\text{score}(q, d) \;=\; E(q) \cdot E(d), \qquad E : \text{text} \to \mathbb{R}^{384}$$

One encoder, applied separately to the query and to the document. Vectors normalised, so the inner product is a cosine.

> **WHY "SEPARATELY" IS THE WHOLE DESIGN**
>
> $E(d)$ does not depend on the query. So **every document can be encoded once, offline, and stored**; at request time you encode one short query and take 5,183 inner products. The expensive part happens before anyone asks.

Hold on to that sentence. It is also the reason the bi-encoder is the *weaker* of the two neural models in this lecture.

One encoder or two?

	Shared encoder	Two encoders
Parameters	one set	two sets
Suits	symmetric tasks — sentence similarity	asymmetric — short query, long document
Risk	a short query and a long abstract are not the same kind of text	twice the parameters on the same labels

We use a shared encoder today, because the model we borrow was trained that way. It is a real modelling choice and it should be reported — a paper that says “bi-encoder” without saying which has told you less than it thinks.

The two architectures, side by side

B Bi-encoder

$E(q)$ and $E(d)$ computed apart, then a dot product. $E(d)$ is query-independent, so it can be stored. The query never touches the document's tokens.

X Cross-encoder

one pass over q and d together. Every query token attends to every document token. Nothing can be stored, because nothing exists until the query arrives.

The accuracy difference and the cost difference have the **same cause**: whether the query is present when the document is read. That is worth being able to say in one sentence in the oral.

What that costs to store

	Per document	Whole corpus
Dense vectors, 384-d float32	1,536 bytes	8.0 MB
Inverted index postings	488 bytes	2.5 MB

Three times the storage, and — unlike postings — **every byte of it is touched by every query**: the dense scan is dense. The inverted index skips the documents sharing no term; the dense index cannot skip anything, because every vector has a non-zero dot product with every query.

✗ That is why approximate nearest-neighbour search exists, and it is the last section of this lecture.

Building the dense index

```
D = encode(model, documents)      # once, offline, 5,183 x 384
D = D / np.linalg.norm(D, axis=1, keepdims=True)

def search(query, k=10):
    q = encode(model, [query])[0]
    q /= np.linalg.norm(q)
    s = D @ q                     # every document, exactly
    return np.argsort(-s)[:k]
```

Six lines, and it is a complete retrieval system. Note what is *not* here: no index structure, no library, no database — the matrix product *is* the search, and on this corpus it is fast enough to be exact.

Everything a vector database adds is a way of not doing that matrix product in full, and it is bought with recall.

The dimension is a dial too

384 dimensions is a choice the model made for us. Its consequences are arithmetic:

Bytes per document	1,536
This corpus	8.0 MB
The same corpus at 768 dimensions	15.9 MB
Ten million documents at 384-d	15.4 GB

Every one of those bytes is read on every exact query. This is the sense in which a dense index is expensive: not to build, but to *use* — and it is what quantisation and approximate search exist to reduce.

Training it: what the objective must say

We want $E(q) \cdot E(d^+)$ large and $E(q) \cdot E(d^-)$ small. Written as a softmax over one positive and a set of negatives:

$$\mathcal{L} = -\log \frac{\exp(E(q) \cdot E(d^+)/\tau)}{\exp(E(q) \cdot E(d^+)/\tau) + \sum_{d^-} \exp(E(q) \cdot E(d^-)/\tau}$$

Which is the cross-entropy of Lecture 17 with the documents as classes, and the contrastive objective you will meet again in Lecture 23 with images on one side.

The positives come from the judgements. **Where do the negatives come from?** There are millions of documents and you cannot score them all.

In-batch negatives

The trick: in a batch of B query–document pairs, every *other* document in the batch is a negative for every query. One encoding pass, B^2 scores.

Batch B	Texts encoded	Scores available	Negatives per query
8	16	64	7
32	64	1,024	31
128	256	16,384	127
512	1,024	262,144	511

Encoding grows linearly and negatives grow linearly, but the *scores* grow quadratically — the dot products are free next to the encoder. This is why dense-retrieval papers report batch sizes in the thousands, and why the batch size is a modelling decision here rather than a memory setting.

The temperature, and what it does

The τ in the objective is not decoration. Scores are cosines, so they live in $[-1, 1]$; a softmax over that range is nearly uniform, and the gradient nearly vanishes.

- Small τ — sharpens the distribution, so the hardest negative dominates the gradient
- Large τ — flattens it, so all negatives contribute nearly equally and the model learns slowly

Typical values are 0.01 to 0.05, which are *small*: dividing a cosine by 0.05 spreads $[-1, 1]$ across $[-20, 20]$, which is where a softmax has gradient to give.

Lecture 23 derives this properly, for the same objective with images on one side. Today it is a parameter with a stated job.

Why the negatives are most of the work

A random negative is almost always trivially wrong: a random abstract shares nothing with the claim, and the model learns to separate “about medicine” from “about anything else”.

- **Hard negatives** — documents BM25 ranks highly but which are not relevant. These teach the distinction that matters
- **False negatives** — and the trap: a hard negative mined this way is often *relevant but unjudged*, so training actively teaches the model that a correct answer is wrong

THE SAME POOLING PROBLEM, NOW INSIDE TRAINING

Lecture 19 said unjudged documents are scored as irrelevant at *evaluation*. Here they are scored as irrelevant during **training**, which is worse: the error is no longer an accounting mistake, it is a gradient.

The false-negative problem, sized

Hard negatives are mined by taking a first stage's top results and calling the unjudged ones negative. On this corpus:

Claim–abstract pairs in the grid	1.55 million
Pairs judged relevant	339
Relevant per claim, mean	1.13

BM25's top 10 for a claim contains, on average, **0.86 judged-relevant abstracts** and 9.14 others. Those others are *unjudged*, not *judged irrelevant* — and on a corpus this sparsely judged, some of them are relevant.

✗ Train on them as negatives and the gradient pushes the model away from correct answers. This is the single commonest reason a fine-tuned retriever underperforms the paper it copied.

Today we do not train one

SciFact has 300 test claims and a training set of a few hundred more. Fine-tuning a bi-encoder on that would measure the split, not the method.

So we take a model trained on something else entirely — `all-MiniLM-L6-v2`, trained on a billion general sentence pairs, never shown a scientific claim — and apply it **zero shot**.

> **WHICH IS ALSO THE HONEST QUESTION**
>
> “Should I put an off-the-shelf embedding model in front of my search?” is the decision most of you will actually face. It is not “can a dense retriever be trained to beat BM25 given enough labels?” — the answer to that is yes, and it is not usually the question you have.

What “trained on a billion pairs” means

`all-MiniLM-L6-v2` was trained by contrastive learning on about a billion sentence pairs scraped from the web: questions and answers, titles and bodies, duplicate posts.

- Its notion of “these two texts belong together” comes from that mixture, and from nothing else
- Scientific claims and abstracts are not in it in any quantity
- 384 dimensions and six layers — small, which is why the notebook runs on a CPU

> **THE PREDICTION TO MAKE BEFORE THE MEASUREMENT**
>
> A model whose training distribution excludes your domain should do worst exactly where the domain vocabulary is most specialised. Write down whether you expect it to beat BM25 here, before the next section.

What the bi-encoder gave up

A single 384-dimensional vector must stand for a 225-token abstract, and it is computed **before the query is known**.

- An abstract about three things compresses to one point, so it is somewhat about each and exactly about none
- Nothing in the encoder can emphasise the part of the document the query is asking about — it has not been asked yet
- Exact term matching is not guaranteed: two different gene names may land close together

✗ That is the price of indexability, and it is the whole reason a second, non-indexable model appears later in this lecture. Notice the trade is *structural*: it is not fixed by a bigger encoder.

Adding one document

	Inverted index	Dense index
New document	append to its terms' postings — microseconds	one encoder pass, then append the vector
New model version	nothing to do	✗ re-encode the entire corpus
Mixed versions	impossible to have	✗ silently wrong — vectors from two models are not comparable

✗ **The last row is a real outage, and a quiet one: nothing errors, the scores are simply meaningless for the documents encoded by the old model. A dense index needs a model version stored beside it, and that is an operational cost the accuracy table never shows.**

The protocol, stated before the numbers

- Same corpus: 5,183 abstracts. Same 300 test claims. Same judgements
- Same metric code — the functions written in last week's notebook, unchanged
- Both retrievers scored **exactly**: every document for every query, no approximate index in either
- The metric was named before the run: **NDCG@10**, with recall@100 reported alongside because the first stage is judged on it

> **WHY THIS SLIDE EXISTS**
>
> Naming the metric after seeing the scores is the commonest dishonesty in this literature, and this lecture is about to produce a result where the temptation is real.

Prediction, before the section that follows

Everything is now in place: the corpus, the model, the protocol, the baseline. Before the next slide, commit to an answer.

> **WRITE DOWN A NUMBER**
>
> BM25 scored **0.6611** NDCG@10 with no training. A general-purpose neural encoder, trained on a billion web sentence pairs and never shown a scientific claim, applied zero-shot to this corpus, will score …?

A prediction made before the measurement is worth ten made after it — and the point of this one is not whether you are right. It is to notice, on the next slide, whether you would have *argued* for the answer if you had seen it first.

MEASURED

Against BM25, on the same judgements

20 minutes

The result

Metric	BM25	Dense, zero shot
NDCG@10	**0.6611 ✓**	**✗ 0.6451**
Precision@1	**0.5433 ✓**	**✗ 0.5033**
MRR	**0.6350 ✓**	**✗ 0.6113**
Recall@10	0.7784	**0.7833 ✓**
Recall@100	0.8852	**0.9250 ✓**

✗ The neural model loses on the headline metric. **Thirty years of age, no training, no GPU, no vector store — and BM25 is ahead by 0.0160 NDCG@10.**

Sit with that for a moment

- It is not a bug in the experiment. Same corpus, same 300 claims, same judgements, same metric code
- It is a well-known result: BEIR was built to report it, and out-of-domain dense retrieval loses to BM25 on a good fraction of its tasks
- Scientific text is exactly where it should be worst — the vocabulary is technical, and a general-purpose encoder has never seen most of it

> **THE TRANSFERABLE LESSON**
>
> "Newer" is not an argument, and neither is "neural". **The baseline you skipped is the result you will not be able to defend.** If BM25 is missing from a retrieval table, that is now a question you know how to ask.

What this result does not license

- It does **not** say dense retrieval is worse in general. Fine-tuned on in-domain pairs, it beats BM25 on this corpus, and that is a well-replicated result
- It does **not** say embeddings are a bad idea. Recall@100 went up, and the mismatch case went from rank 4,999 to 8
- It *does* say that this particular substitution — off-the-shelf encoder for BM25, no labels, technical corpus — is a loss, and that is a decision many teams make without measuring

Stating the scope of a claim is not hedging. A result whose boundary you cannot draw is a result you cannot use.

But look at recall@100

Metric	BM25	Dense	Dense − BM25
NDCG@10	0.6611	0.6451	✗ **−0.0160**
Recall@10	0.7784	0.7833	+0.0049
Recall@100	0.8852	0.9250	**+0.0398** ✓

The dense retriever is *worse at ordering* and **better at finding**. It retrieves 92.5% of relevant abstracts somewhere in its first hundred, against BM25's 88.5%.

Recall from the last lecture what a first stage is judged on: not what the user sees, but what it makes possible. On *that* criterion — the criterion for the job it would actually be given — the dense retriever wins.

Do they find the same documents?

Take every relevant abstract in the test set and ask which method placed it in the top ten:

Found by	Count	Share
Both	227	67.0%
BM25 only	**30 ✓**	8.8%
Dense only	**38 ✓**	11.2%
Neither	**✗ 44**	13.0%

68 of 339 relevant abstracts are found by exactly one of the two. They are not two attempts at the same ranking — they fail on different documents.

Two methods with almost the same score and different errors is the classic signal that combining them will help. Which we will do, after the interesting case.

A mean hides the disagreement

BM25 0.6611, dense 0.6451 — a gap of 0.0160, which sounds like two systems doing almost the same thing.

The overlap table says otherwise: 227 relevant abstracts found by both, 30 by BM25 alone, 38 by the dense model alone. Per claim, they frequently disagree completely.

THE GENERAL POINT

Two systems with the same mean can have entirely different error sets. The mean answers “which is better on average”; it cannot answer “are these the same system”, and only the second question tells you whether to combine them.

FAILURE CONDITION — A MEAN OVER QUERIES

The two systems fail on different documents: 68 of 339 relevant abstracts were found by exactly one of them. A mean answers which is better on average and cannot answer whether they are the same system — and only the second question tells you whether fusing them is worth anything.

The claim BM25 could not find

“In mice, P. chabaudi parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers.”

Rank of its one relevant abstract	
BM25	✗ 4,999
Dense, zero shot	8 ✓

From 4,999 to 8. The abstract still shares almost no words with the claim — nothing about the text changed. What changed is that the comparison is no longer between strings.

This is the case the whole lecture was built around, and it is worth being precise about what it does and does not show: it does **not** show the dense retriever is better. The table three slides ago showed it is not. It shows the two methods fail *independently*.

LAST THIRTY MINUTES

Combining them, and paying for it

30 minutes

Hybrid retrieval: fuse the two rankings

Two methods, complementary errors. The scores are not on a comparable scale — BM25 returns unbounded sums, the dense model returns cosines — so fuse the **ranks** instead:

$$\mathrm{RRF}(d) = \sum_{\text{systems}} \frac{1}{k + \mathrm{rank}(d)}, \qquad k = 60$$

Reciprocal rank fusion. No training, no tuning, no calibration — and it needs nothing from the two systems but their orderings, which is why it is what people actually deploy.

The constant k damps the top ranks so one system cannot dominate on a single confident hit. 60 is conventional, and yes, that is the same kind of convention as $\log_2(i+1)$.

Hybrid, measured

Metric	BM25	Dense	Hybrid
NDCG@10	0.6611	0.6451	**0.6948** ✓
Precision@1	0.5433	0.5033	**0.5633** ✓
Recall@10	0.7784	0.7833	**0.8239** ✓
Recall@100	0.8852	0.9250	**0.9617** ✓

Better than either, on every metric, from adding two reciprocals. The gain over BM25 is **+0.0338** NDCG@10 — larger than anything else in this lecture will buy you.

And it is not mysterious: the earlier table said 68 relevant abstracts were found by exactly one system. Fusion is how you keep both.

Why not just add the scores?

The obvious fusion is $\alpha \cdot \mathrm{BM25} + (1 - \alpha) \cdot \mathrm{cosine}$. It works, and it costs you two things:

- The scales are incomparable and corpus-dependent — BM25 is an unbounded sum, the cosine is in $[-1, 1]$ — so the scores must be normalised, and the normalisation is another choice
- α must be tuned on held-out queries, so you now need labels for a method whose selling point was needing none

Rank fusion avoids both: ranks are already comparable, and there is nothing to fit. Score fusion can do better when you have the labels to tune it — which is exactly the condition under which you would have fine-tuned the encoder instead.

The other neural model: cross-encoders

$$\text{score}(q, d) = \text{Transformer}\big(\,[\text{CLS}]\ q\ [\text{SEP}]\ d\,\big)$$

Concatenate the query and the document and put **both through the model together**. Every token of the query can attend to every token of the document, which is exactly what a bi-encoder forbids by construction.

	Bi-encoder	Cross-encoder
Sees	each text alone	the pair, jointly
Document vectors	precomputed	impossible — depends on the query
Cost per query	one encoding + N dot products	N transformer passes
Accuracy	lower	higher

Why the accurate model cannot go first

A full cross-encoder scan of this corpus, for these claims, is

$$5,183 \times 300 = 1,554,900 \text{ transformer passes}$$

On the *toy* corpus of this course. A web index is 10^{10} documents and the user is waiting 200 ms, so the arithmetic is not close — it is off by ten orders of magnitude.

HENCE THE TWO-STAGE PIPELINE OF LECTURE 19

A **cheap first stage** reduces millions to a hundred; an **expensive second stage** reorders that hundred. The architecture of every production search system is a direct consequence of the fact that the good model does not scale.

What the second stage costs, per query

Stage	Model passes per query
BM25	0
Bi-encoder	1 (the query)
Cross-encoder over top 10	10
Cross-encoder over top 100	100
Cross-encoder over the corpus	✗ 5,183

The last row is why the pipeline exists, and the middle rows are why its depth is the parameter you actually argue about in a design review. Both are the same arithmetic.

An accurate model that cannot be indexed

FAILURE CONDITION

The cross-encoder reads query and document together, which is exactly why it is stronger and why it cannot be precomputed. Put it first and the latency is the corpus size; the quality number says nothing about whether the system can be run.

Re-ranking, measured

Cross-encoder over BM25's top k, everything below k left where it was:

System	NDCG@10	P@1	Pairs scored
BM25 alone	0.6611	0.5433	0
+ re-rank top 10	0.6656	0.5600	3,000
+ re-rank top 50	0.6773	0.5667	15,000
+ re-rank top 100	**0.6781**	**0.5667**	**30,000**

Going from 10 to 50 buys +0.0117; going from 50 to 100 buys +0.0008 for twice the compute. **The depth is a dial with diminishing returns, and it is the main thing you tune in a deployed system.**

Reading the re-ranking curve

Depth	NDCG@10	Gain	Passes
0	0.6611	—	0
10	0.6656	+0.0046	10
50	0.6773	+0.0117	50
100	0.6781	+0.0008	100

The first ten candidates give most of the gain; the next ninety give a tenth of it, for ten times the compute. That is the shape of every re-ranking curve you will meet, and the reason production depths are usually in the tens rather than the thousands.

Note also that recall@10 barely moves: re-ranking the top 10 cannot change *which* documents are in the top 10 at all. It changes their order, and P@1 with it.

The ceiling the re-ranker cannot pass

Re-ranking reorders; it cannot retrieve. So the recall of the first stage at depth k is a hard bound on the second stage at depth k:

Depth k	10	50	100	1000
BM25 recall@k	0.7784	0.8654	0.8852	0.9650

Re-rank BM25's top 100 and a *perfect* cross-encoder still cannot exceed recall 0.8852 at any cutoff — 11.5% of relevant abstracts are simply not in the candidate set.

✗ Notice that the hybrid first stage reaches recall@100 of 0.9617. Improving the first stage raises the ceiling for everything above it; improving the second stage never does. That is the single most useful sentence in this lecture for building a system.

An exam-style question, worked

“A system reports NDCG@10 of 0.6781 using a cross-encoder over the top 100 of a BM25 first stage. Its authors propose replacing the cross-encoder with a larger one. What is the most you can say about the gain, and what would you propose instead?”

- The first stage’s recall@100 is 0.8852, so **no** re-ranker can exceed that recall — 11.5% of relevant documents are absent from the candidates
- Going from depth 50 to 100 already bought only +0.0008, so the candidates are close to exhausted
- Propose improving the **first stage**: a hybrid one reaches recall@100 0.9617, which raises the ceiling itself

Section B of the paper is exactly this shape: a table, a proposal, and the question of whether the proposal can possibly work.

Approximate nearest neighbours: the last cost

One problem remains. The dense scan touches every one of the 8.0 MB of vectors, for every query, and unlike the inverted index it cannot skip anything.

The standard answer, and the one we build in the notebook: **cluster the vectors once** (k-means, 64 clusters), then at query time score only the nearest few clusters.

NOTE WHAT THIS CHANGES ABOUT THE NUMBERS

Every dense number so far was an *exact* scan, so it measured the model. From here on the numbers measure the model **and the index**, and the two must be reported separately — which is exactly why the whole lecture was built on a corpus small enough to scan exactly.

FAILURE CONDITION — AN INDEX THAT QUIETLY STOPS BEING EXHAUSTIVE

An approximate index trades recall for speed, and the recall it gives up is not reported anywhere: the query returns k neighbours as always. The ranking metric then scores a system whose candidate list is already missing documents, and blames the model.

Why clustering works at all

The claim behind an IVF index: a query's nearest documents are *mostly* in the clusters nearest the query. Which is a claim about geometry, and it is only approximately true.

- A document near a cluster boundary is missed when its cluster is not probed — and boundary documents are common in high dimensions
- So the index has a **recall**, and the recall is a parameter, not a property

> **THE MEASUREMENT THAT MATTERS**
>
> Recall of the approximate index is measured **against the exact scan**, never against the relevance judgements. Mixing the two conflates "my index missed it" with "my model ranked it low", and they have different fixes.

The recall/latency trade-off, measured

Recall against the *exact* dense ranking, not against the judgements — this table is about the index, not the model:

Clusters probed	Corpus scanned	Recall@10 vs exact
1	1.8%	0.5047
2	3.6%	0.6580
4	**7.3%**	**0.8013**
8	14.1%	0.8920
16	27.9%	0.9577
64	100.0%	1.0000

Probing 4 of 64 scans **7.3%** of the corpus and keeps **80.1%** of the exact top ten. A dial, not a default — and a vector database ships with it set by someone who never saw your corpus.

A latency budget, worked

Suppose 200 ms end to end, and take the pipeline as built:

Stage	Model passes	Scales with
BM25	0	query terms × postings length
Encode the query	1	nothing — the query is short
Dense scan (exact)	0	**✗ corpus size × 384**
Dense scan (probe 4 of 64)	0	7.3% of that
Re-rank top 50	50	the depth you chose

Two of the five rows are dials you set, and both trade quality for time along a curve you can measure. The budget does not tell you where to sit on them — but it does tell you that the choice is yours and has to be made deliberately.

The whole table, in one place

System	NDCG@10	R@100	Cost per query
Corpus unranked	✗ **0.0012**	0.0117	nothing
BM25	0.6611	0.8852	postings walk
Dense, zero shot	0.6451	0.9250	1 encode + full scan
BM25 + cross-encoder@100	0.6781	0.8852	+ 100 transformer passes
Hybrid (BM25 + dense, RRF)	**0.6948**	**0.9617**	**1 encode + both**

The best NDCG@10 in this lecture comes from the row with **no training in it at all** — two untrained retrievers and a sum of reciprocals.

The arc of the two IR lectures

Step	Because
Boolean retrieval	obvious — and returns nothing for 98% of claims
Score, do not filter	a missing term should cost something, not everything
idf, saturation, length	three named failures of raw counting
Dense retrieval	strings cannot capture paraphrase
Hybrid	the two fail on different documents
Re-ranking	the accurate model does not scale
Approximate index	the dense scan cannot skip anything

Every row is a response to a measured failure of the row above. That is the shape this course keeps asking you to produce.

The two lectures in one table

System	NDCG@10	R@100	Trained?
Corpus unranked	0.0012	0.0117	no
Raw term counting	0.0944	—	no
BM25	0.6611	0.8852	no
Dense, zero shot	0.6451	0.9250	elsewhere
BM25 + cross-encoder@100	0.6781	0.8852	elsewhere
Hybrid	**0.6948**	**0.9617**	**no**

Six systems, one corpus, one metric named in advance. That table is the deliverable of Part V's first half.

How to choose, in practice

Situation	What to do
No labels, technical vocabulary	BM25. Measure before you consider anything else
No labels, everyday language, paraphrase matters	hybrid — BM25 plus an off-the-shelf encoder, fused
Thousands of labelled pairs in your domain	fine-tune a bi-encoder; now it can beat BM25
Latency budget allows a second stage	add a cross-encoder over the top 50–100
Corpus too large to scan	approximate index, and report the recall you chose

✗ Every row starts with BM25 in the table. Not because it wins — on your corpus it may not — but because a comparison without it is not a measurement.

Five questions, for a neural retrieval claim

1. **Is BM25 in the table?** On the same corpus, queries and judgements, tuned no less carefully than the neural model
2. **Zero-shot or fine-tuned?** And if fine-tuned, on what, and how close is it to the test corpus?
3. **Exact scan or approximate index?** If approximate, at what recall against the exact scan?
4. **Where did the negatives come from?** And what fraction of them are unjudged rather than judged irrelevant?
5. **First stage or whole pipeline?** A re-ranker's NDCG is a property of the candidates it was given

What this lecture left out

- **Late interaction** — keep one vector per token and score with a max-similarity sum. Between the two architectures in both cost and accuracy
- **Learned sparse retrieval** — use a transformer to *predict term weights*, then use an ordinary inverted index. The two families are not as separate as this lecture made them look
- **Distillation** — train the bi-encoder to imitate the cross-encoder, moving accuracy into the model that can be indexed

Not examinable, and each is a direct response to a trade-off this lecture measured. If Part V interests you, these three are where to read next.

Where students lose marks on this material

- Saying a cross-encoder is “a bigger bi-encoder”. It is a different computation: joint, and therefore unindexable
- Claiming a re-ranker improves recall@100. It cannot — it reorders a fixed candidate set
- Reporting approximate-index recall against the *judgements* rather than against the exact scan, which conflates two errors
- Describing in-batch negatives as “free”. The scores are free; the encodings are not — and since B sets how many negatives each query is scored against, the batch size is a modelling decision, not just a memory setting
- Concluding from this lecture that dense retrieval does not work. The claim is narrower: *zero-shot*, on *this* corpus

Reading

- **These lecture notes are the primary source** — lecture-20.pdf, the extended notes for this lecture. The textbook does not cover this material
- Karpukhin et al., *Dense Passage Retrieval* — where in-batch negatives were made to work
- Thakur et al., *BEIR* — the benchmark built to test zero-shot generalisation, and the source of the result we reproduced
- Nogueira and Cho, *Passage Re-ranking with BERT* — the cross-encoder as a second stage
- Johnson et al., *Billion-scale similarity search* — what an IVF index is, done properly

Examinable content is what is on these slides and in the notebook.

The notebook for this lecture

Open Lecture 20 in Colab

- **Runs on free CPU**, in about ten minutes. Encoding the corpus once is most of it; nothing is trained
- The same BM25 code as Lecture 19, so the baseline in the comparison is the baseline you already built
- RRF, cross-encoder re-ranking, and an IVF index written out by hand in twenty lines — no vector database

> **WHAT TO CHANGE**
>
> Re-rank the *hybrid* top 100 rather than BM25's. The ceiling slide predicts what should happen to recall; check whether NDCG follows.

Vocabulary

Bi-encoder	query and document encoded separately; score is a dot product
Cross-encoder	the pair encoded jointly; cannot be precomputed
In-batch negatives	the other documents in the batch, used as negatives
Hard negative	a high-scoring irrelevant document, mined from a first stage
First stage / re-ranker	cheap over everything; expensive over a few
RRF	fusing rankings by summing $1/(k + \mathrm{rank})$
ANN, IVF, nprobe	approximate search; cluster the vectors, probe a few
Zero shot	applied without training on this task or corpus

Scope

Scope

- vocabulary mismatch and why lexical repairs are bounded; what a bi-encoder computes and why it is indexable; the contrastive objective and in-batch negatives; why hard negatives matter and how they go wrong; cross-encoders and the first stage/re-rank argument including the ceiling; hybrid fusion; the recall/latency trade-off of an approximate index — EXAMINABLE
- specific model checkpoints, vector-database APIs, HNSW and product quantisation internals — NOT EXAMINABLE — ENGINEERING
- late interaction (ColBERT), learned sparse retrieval (SPLADE), distillation of cross-encoders into bi-encoders — BEYOND THE SYLLABUS

Summary

1. A lexical index matches strings; relevance is about meaning, and no patch on the index closes that gap
2. A bi-encoder can be indexed because $E(d)$ does not depend on the query — and that same independence is why it is the weaker model
3. Zero shot on this corpus, it **loses** to BM25 on NDCG@10 (0.6451 against 0.6611) and **wins** on recall@100 (0.9250 against 0.8852)
4. They fail on different documents: 68 of 339 relevant abstracts were found by exactly one, so fusing them beats both (0.6948)
5. The accurate model goes second because it does not scale; and it cannot exceed the first stage's recall at that depth

Where this sits in the course

Lecture 8	a vector space, and distance in it — here, the score itself
Lecture 17	cross-entropy — here, over documents instead of classes
Lecture 18	the encoder producing both vectors
Lecture 19	the metrics, the baseline, and the judgements
Lecture 23	the same contrastive objective, with images
Lecture 24	this retriever, feeding a generator

Lecture 24 is worth noting now: retrieval-augmented generation is *this pipeline* with a language model on the end, and its failures are mostly this lecture's failures.

One line to keep

> THE LINE
>
> **The first stage sets the ceiling; the second stage only decides how close you get to it.**

Which is why the interesting engineering is almost always in the cheap model, not the good one — and why the largest single gain in this lecture, +0.0338 NDCG@10, came from fusing two untrained first stages rather than from any neural model.

If you take one table away

	NDCG@10	Recall@100
BM25	**0.6611 ✓**	0.8852
Dense, zero shot	0.6451	0.9250
Both, fused	**0.6948**	**0.9617**

The old method wins the ranking. The new method wins the retrieval. **Neither is the answer, and the combination beats both on both** — for the cost of adding two reciprocals.

If a lecture in this part has a single practical recommendation, it is that one: before you replace a lexical retriever, try keeping it.

Before the next lecture

- Run the notebook, and try the change on the notebook slide: re-rank the hybrid candidates instead of BM25's. Predict the direction before you run it
- Part V continues with **recommendation**, which is retrieval with the query removed — there is no query, only a user and what they did
- Keep the evaluation habits of Lecture 19. Recommender systems are where they are most often abandoned, and Lecture 22 is largely about that

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 20

five, in the style of the written paper

Exercises — Lecture 20

1 Explain why a bi-encoder can be indexed and a cross-encoder cannot, and say what the two differences have in common as a cause. [5]

2 A first stage has recall@100 of 0.885. State the highest recall a perfect re-ranker over those 100 candidates can reach, and what follows for where to spend effort. [4]

3 In-batch negatives are described as free. State precisely what is free and what is not, and why the batch size is a modelling decision rather than a memory setting. [5]

Solutions on Lecture 21’s deck.

Exercises — Lecture 20 (continued)

4 Hard negatives are mined from a first stage's top results. State the trap, and why it is worse at training time than at evaluation time. [5]

5 An approximate index reports recall 0.80. State what that number must be measured against, and what is conflated if it is not. [4]

Solutions on Lecture 21's deck.

THE FIVE SET AT THE END OF LECTURE 19

Solutions Lecture 19

not part of this lecture

Solutions — Lecture 19

1. A ranking has relevant documents at ranks 1, 3 and 10, with three relevant documents in total. Compute AP and…

✓ AP $= \frac{1}{3}\left(1 + \frac{2}{3} + \frac{3}{10}\right) = 0.6556$**; NDCG@10** $= \frac{1+0.5+0.2891}{1+0.6309+0.5} = 0.8396$**.**

- AP averages the precision at each hit and divides by $|R|$, not by the number of hits found
- DCG discounts by $1/\log_2(i+1)$, and the ideal puts all three at ranks 1–3

2. State why $\log_2(i+1)$ is used rather than $\log_2(i)$ in DCG, and whether the choice of discount is derived…

✓ $\log_2(1) = 0$ **would divide by zero at rank 1. The discount is conventional.**

- The $+1$ is a necessity, not a preference
- The shape encodes a belief about attention, and reporting NDCG adopts it

Solutions — Lecture 19 (2)

3. BM25 has three parts. Name the failure of raw term counting each one answers, then say which part the…

✓ idf answers common words dominating; saturation answers repetition; length normalisation answers long documents. Of those removed singly idf costs most; saturation is never removed on its own.

- Removing idf alone costs 0.1261 NDCG@10 and removing length normalisation alone costs 0.0120 — those are the two the ablation isolates
- The remaining row drops saturation and length together for 0.1809, and a measurement that removes two things at once cannot attribute the loss to either of them

4. 98% of claims match no document under Boolean conjunction. State the property of the conjunction that causes…

✓ One missing term excludes the document entirely. Score rather than filter.

- The conjunction is brittle, and an empty result looks like 'no such document exists'
- Scoring makes a missing term cost something rather than everything

Solutions — Lecture 19 (3)

5. Relevance judgements are made by pooling. State what happens to a document no pooled system ranked highly,…

✓ It is unjudged and scored as irrelevant, so a new method is penalised for finding it.

- Everything outside the pool is treated as not relevant by convention
- Gains on a mature benchmark are partly gains at matching the pool

LECTURE 21 · LECTURE NOTES

Recommender systems: from ratings to factors

No query, and feedback that is a by-product of use
Explicit ratings against implicit feedback, and why implicit data has no negatives
matrix factorisation, and why the SVD of Lecture 8 cannot simply be taken

Part V · Lecture notes · examinable